

CSS Notes

Quick reference and study guide for core CSS concepts

1. What Is CSS?

CSS (Cascading Style Sheets) controls the look and layout of HTML content. HTML gives structure, while CSS handles presentation such as colors, spacing, typography, and responsive behavior.

- CSS stands for Cascading Style Sheets.
- It separates content (HTML) from design (CSS).
- One stylesheet can style multiple pages for consistency.

2. Basic CSS Syntax

A CSS rule has a selector and a declaration block. Each declaration is a property-value pair.

```
selector {  property: value;  }  
h1 {      color: #1e40af;    font-size: 2rem; }
```

3. Ways to Add CSS

Inline CSS

```
<p style="color: red;">Hello</p>
```

Internal CSS

```
<style>  p { color: red; } </style>
```

External CSS (recommended)

```
<link rel="stylesheet" href="styles.css" />
```

4. Selectors

- Element selector: `p { ... }`
- Class selector: `.card { ... }`
- ID selector: `#header { ... }`
- Descendant selector: `nav a { ... }`
- Pseudo-class: `a:hover { ... }`
- Pseudo-element: `p::first-line { ... }`

5. The Cascade, Specificity, and Inheritance

- Cascade: later matching rules can override earlier ones.
- Specificity: ID > class/pseudo-class > element.
- Inheritance: some properties (like color and font) pass from parent to child.

```
/* Specificity example */ #title { color: red; } .title { color: blue; } h1  
{ color: green; }
```

6. Box Model

Every element is a box made of content, padding, border, and margin.

- content: actual text or media
- padding: inner space around content
- border: line around padding/content
- margin: outer space between elements

```
* { box-sizing: border-box; }
```

7. Units and Sizing

- px: fixed pixel value
- %, vw, vh: relative to container/viewport
- em: relative to parent font-size
- rem: relative to root font-size (preferred for scalable typography)

8. Layout Fundamentals

Display

- block: starts on a new line and takes full width
- inline: flows within text and ignores width/height
- inline-block: inline flow with block sizing

Positioning

- static, relative, absolute, fixed, sticky

Flexbox

```
.container { display: flex; justify-content: space-between; align-items: center; gap: 1rem; }
```

Grid

```
.grid { display: grid; grid-template-columns: repeat(3, 1fr); gap: 1rem; }
```

9. Responsive Design

- Use fluid units and flexible layouts.
- Add media queries for breakpoints.

```
@media (max-width: 768px) { .grid { grid-template-columns: 1fr; } }
```

10. Typography and Colors

- font-family, font-size, font-weight, line-height are key text properties.
- Use color formats like hex, rgb(), and hsl().
- Maintain good contrast for readability and accessibility.

11. Transitions and Animations

```
.btn { transition: background-color 0.3s ease; } .btn:hover {  
background-color: #1d4ed8; }  
  
@keyframes fadeIn { from { opacity: 0; } to { opacity: 1; } }
```

12. CSS Best Practices

- Keep styles modular and reusable (utility classes or component-based naming).
- Prefer external stylesheets over inline styles.
- Use meaningful class names (e.g., .product-card instead of .blue-box).
- Avoid over-specific selectors to keep CSS easy to maintain.
- Group related rules and comment complex sections.

13. Quick Revision Checklist

- I can explain cascade and specificity.
- I can use selectors (class, id, descendant, pseudo-classes).
- I understand the box model and box-sizing.
- I can build layouts with flexbox and grid.
- I can make a page responsive with media queries.

End of CSS Notes. Practice by styling small projects daily and iterating on layout, spacing, and readability.